

Server configuration

Configuración de servicios en las aplicaciones web

Ahora pasamos a la configuración de servidores, métodos http, cabeceras de seguridad, etc. Todo es parte de la fase de **enumeración** y **análisis** para entender la superficie de ataque de una aplicación.

La Conexión entre Conceptos

Imagina que estás auditando una aplicación web. Tu primer objetivo es recopilar la mayor cantidad de información posible sobre ella, lo cual se divide en un escaneo pasivo y uno activo.

- **Escaneo pasivo:** Aquí buscas pistas sobre el objetivo sin contactarlo directamente. Te apoyas en fuentes públicas como motores de búsqueda y bases de datos de filtraciones de datos ([leaks](#)) para descubrir subdominios o credenciales expuestas.
- **Escaneo activo (o *fingerprinting*):** Una vez que tienes una idea del objetivo, lo contactas directamente. Aquí es donde entra **Nmap**. Con Nmap, puedes escanear la dirección IP de la aplicación para ver qué puertos están abiertos y qué servicios se están ejecutando. El objetivo no es solo encontrar el puerto 80 (HTTP) o 443 (HTTPS) , sino también detectar la versión del servidor web (por ejemplo, Apache 2.4.25). Esta información es crucial porque te permite buscar vulnerabilidades públicas asociadas a esa versión.

Luego, el análisis se vuelve más granular:

- **Análisis de recursos y archivos:** Ahora que conoces el servidor, buscas archivos expuestos. Esto incluye archivos de infraestructura como [robots.txt](#) y [sitemap.xml](#), que te dan un mapa de la estructura de la

aplicación y sus directorios. También buscas copias de seguridad (`backup.db`, `files.old`) que, si están accesibles, podrían contener credenciales o código fuente.

- **Análisis de la configuración del servidor web:** La configuración del servidor determina cómo maneja las peticiones. Aquí es donde los **métodos HTTP** y las **cabeceras de seguridad** entran en juego. Un servidor web mal configurado podría permitir métodos inseguros (`PUT`, `DELETE`) o tener cabeceras de seguridad débiles que dejan al usuario vulnerable a ataques como el *clickjacking*.

En resumen, todo está interconectado. La información que obtienes en cada fase (del servidor, de los archivos, de las configuraciones) te lleva a la siguiente, permitiéndote construir un modelo de ataque coherente. La relación es que, al auditar, pasas de una vista general de la red y sus servicios a un análisis minucioso de cómo se comporta la aplicación web, lo cual te permite identificar fallos de seguridad muy específicos.

En esta sesión vamos a comenzar a hablar de algunos factores que debemos tener en cuenta cuando nos encontremos analizando la configuración de un servidor.

Para ello vamos a comenzar por conocer qué servicios y qué ficheros, qué rutas debemos prestar atención para identificarlos y analizarlos posteriormente.

En primer lugar, lo que debemos tener claro es que en el caso de las aplicaciones web (web ressources), los puertos que están asociados a estos servicios son el 80 para el caso del protocolo HTTP y 443 en el caso del protocolo HTTPS.

De la misma forma es posible usar puertos alternativos, ya sea porque los puertos anteriores a 443 están dando servicio a otras aplicaciones o también puede ser porque las aplicaciones que queramos usar en estos puertos alternativos que pueden ser 8080, 8081, 443, queremos darle una orientación a un servicio que sea más de gestión que otras aplicaciones que puedan estar expuestas por defecto en los puertos 80 y 443.

De esta forma estas aplicaciones en estos puertos alternativos también pueden ser tanto como decimos, para gestión como para usuarios que estén limitados, que no sean para todos los públicos, que sean usuarios de un getter pack de un determinado departamento o usuarios concretos.

Nmap

Para identificar estos servicios y realizar este descubrimiento de puertos contamos con la herramienta llamada NMAP, el nombre viene de Network Mapper y es la herramienta por excelencia para el descubrimiento de servicios de protocolos y además tiene una gran variedad de parámetros que nos pueden ayudar a obtener un mayor nivel de detalle de las tecnologías de sus versiones, de los sistemas operativos que hay corriendo por detrás de estos servicios e incluso de las versiones que pueden estar en estos servicios, junto con vulnerabilidades que puedan estar asociadas a estas versiones de las tecnologías que la herramienta de nmap nos identifique.

Nmap (Network Mapper) es una herramienta de código abierto muy popular y versátil que se utiliza para escanear redes y realizar descubrimiento de hosts. En esencia, actúa como un "mapeador" de redes, enviando paquetes a los hosts y analizando sus respuestas para determinar qué dispositivos están en línea, qué servicios están ejecutándose, qué puertos están abiertos y qué sistemas operativos están utilizando.

Parámetros y Casos de Uso Comunes de Nmap

A continuación, tenemos los parámetros más utilizados de **nmap** y su función:

- **-sS**: Realiza un escaneo de tipo **TCP SYN** o "escaneo a medio abrir" (*half-open*). Es muy rápido y discreto, ya que no completa la conexión TCP, lo que lo hace menos propenso a ser detectado por los sistemas de registro del servidor. Es el modo por defecto para los usuarios con permisos de **root**.
- **-sT**: Escaneo de conexión **TCP** completo. Es más lento y ruidoso que el escaneo **SYN** porque establece una conexión completa a cada puerto, lo que

lo hace más fácil de detectar. Es útil cuando no tienes permisos de **root** para realizar un escaneo **SYN**.

- **-sV**: Este es un parámetro de gran valor en la fase de reconocimiento. Se utiliza para la **detección de versiones** de los servicios que se ejecutan en los puertos abiertos. En lugar de solo decirte que el puerto 80 está abierto, te dirá si está ejecutando un servidor web **Apache**, **Nginx** o **Tomcat**, y qué versión específica tiene. Esta información es vital para buscar vulnerabilidades conocidas asociadas a esa versión.
- **-sO**: Escaneo de protocolos IP. Este parámetro no escanea puertos TCP o UDP, sino que identifica qué **protocolos IP** están disponibles en un host, como ICMP, TCP, UDP, etc..
- **-sC**: Este parámetro, conocido como escaneo de **scripts predeterminados**, es una de las funciones más poderosas de **nmap**. Ejecuta una serie de *scripts* de **nmap** para detectar vulnerabilidades comunes, configuraciones incorrectas o para obtener más información de los servicios. Por ejemplo, puede verificar si un servidor web permite ciertos métodos HTTP vulnerables.
- **-T<0-5>**: Controla la velocidad del escaneo.
 - **T0** es el más lento y sigiloso, ideal para evitar sistemas de detección de intrusos.
 - **T5** es el más rápido y ruidoso, pensado para redes internas donde el sigilo no es una prioridad. El valor **T4** es el más utilizado, ya que ofrece un buen equilibrio entre velocidad y discreción.
- **-p**: Especifica los puertos a escanear. Puedes definir un puerto único (**-p 80**), un rango de puertos (**-p 1-1000**) o todos los puertos (**-p 0-65535**), lo cual es útil para encontrar servicios en puertos no estándar.
- **-Pn**: Deshabilita la detección de host (ping). Este parámetro asume que el host está en línea y escanea todos los puertos directamente. Es útil si el host está detrás de un firewall que bloquea los paquetes de ping.

- **-A**: Escaneo agresivo. Es un atajo para habilitar varias opciones avanzadas, incluyendo la detección de sistema operativo (**-O**), la detección de versión (**-sV**), el escaneo de scripts predeterminados (**-sC**) y el **traceroute**.
- **--script <nombre_del_script>**: Permite ejecutar un *script* de **nmap** específico. Los *scripts* de **nmap** son muy versátiles y pueden realizar tareas como la enumeración de vulnerabilidades, la autenticación en servicios, o incluso ataques de fuerza bruta.

Ejemplos Prácticos de Uso

A continuación, se muestran algunos ejemplos de cómo combinar estos parámetros para diferentes escenarios:

Escaneo Básico de Puertos Abiertos:

```
nmap -sV -p 80,443 192.168.1.1
```

- Este comando escanea los puertos 80 y 443 del host **192.168.1.1** y detecta la versión del servicio en cada uno.

Escaneo Completo para un Reconocimiento Exhaustivo:

```
nmap -A -p0-65535 192.168.1.1
```

- Este comando utiliza el escaneo agresivo (**-A**) para obtener el máximo de información posible. Escanea todos los puertos del **0** al **65535** y proporciona detalles sobre el sistema operativo, las versiones de los servicios y los scripts predeterminados.

Uso de Scripts para Encontrar Métodos HTTP:

```
nmap -p 443 --script http-methods --script-args  
http-methods.url-path='/index.php' localhost
```

- Este comando escanea el puerto 443 y utiliza el *script* **http-methods** para verificar qué métodos HTTP (**GET**, **POST**, **PUT**, **DELETE**, **OPTIONS**, etc.) están permitidos en una página específica del

servidor. La salida puede revelar métodos inseguros que un atacante podría explotar.

Para conocer mejor el potencial que tiene esta herramienta, vamos a realizar una pequeña prueba sobre la máquina virtual de Metasploitable con máquina víctima y la máquina virtual de Kali Linux como máquina atacante.

Por una parte veremos aquí la máquina de Metasploitable con la dirección IP 192.168.186.132 y la máquina Kali Linux donde ya tenemos escrito nuestro comando nmap:

```
nmap -sV -vv 192.168.186.132
```

Por una parte podríamos decir de forma básica el comando es `nmap` y la dirección IP o el target que queremos analizar del que queremos descubrir los servicios, pero además nmap tiene una gran multitud de parámetros que nos van a ampliar la información sobre los servicios que encuentre. En este caso el parámetro `-sV` nos va a dar la versión del servicio de cada uno de los servicios que va a ir encontrando y además he puesto el parámetro `-vv` para añadir un mayor nivel de verbose o detalle en las del comando en la respuesta de las pruebas que va analizando esta herramienta.

En un primer momento nos indica los puertos que están abiertos por el servidor de la aplicación y por otra parte realiza una serie de pruebas en las que nos va a identificar tanto el tipo de servicio de tecnología que hay detrás, como en este caso por el parámetro que hemos indicado anteriormente de `-sV`, nos indica también la versión de este servicio y de esta tecnología que está usando.

PORT	STATE	SERVICE	REASON	VERSION
21/tcp	open	ftp	syn-ack ttl 64	vsftpd 2.3.4
22/tcp	open	ssh	syn-ack ttl 64	OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp	open	telnet	syn-ack ttl 64	Linux telnetd
25/tcp	open	smtp	syn-ack ttl 64	Postfix smtpd
53/tcp	open	domain	syn-ack ttl 64	ISC BIND 9.4.2
80/tcp	open	http	syn-ack ttl 64	Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp	open	rpcbind	syn-ack ttl 64	2 (RPC #100000)
139/tcp	open	netbios-ssn	syn-ack ttl 64	Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp	open	netbios-ssn	syn-ack ttl 64	Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp	open	exec	syn-ack ttl 64	netkit-rsh rexecd
513/tcp	open	login	syn-ack ttl 64	OpenBSD or Solaris rlogind
514/tcp	open	tcpwrapped	syn-ack ttl 64	
1099/tcp	open	java-rmi	syn-ack ttl 64	GNU Classpath grmiregistry
1524/tcp	open	bindshell	syn-ack ttl 64	Metasploitable root shell
2049/tcp	open	nfs	syn-ack ttl 64	2-4 (RPC #100003)
2121/tcp	open	ftp	syn-ack ttl 64	ProFTPD 1.3.1
3306/tcp	open	mysql	syn-ack ttl 64	MySQL 5.0.51a-3ubuntu5
5432/tcp	open	postgresql	syn-ack ttl 64	PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp	open	vnc	syn-ack ttl 64	VNC (protocol 3.3)
6000/tcp	open	X11	syn-ack ttl 64	(access denied)
6667/tcp	open	irc	syn-ack ttl 64	UnrealIRCd
8009/tcp	open	ajp13	syn-ack ttl 64	Apache Jserv (Protocol v1.3)
8180/tcp	open	http	syn-ack ttl 64	Apache Tomcat/Coyote JSP engine 1.1

MAC Address: 00:0C:29:31:E7:72 (VMware)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Además también hay otros parámetros como es por ejemplo `-sC` en el que lanza paralelamente a lo largo de este escaneo y de esta enumeración un conjunto de scripts que tiene nmap en su base de datos y nos puede lanzar pruebas dentro de estos servicios.

Identifica para indicarnos recursos que puedan estar por defecto, credenciales que puedan encontrarse por defecto y eso nos va a dar un mayor nivel de conocimiento en estos servicios.

También podemos utilizar el parámetro `-sO` para indicar el tipo de sistema operativo que también puede haber corriendo detrás de los servicios.

Y una gran opción, un gran parámetro que tiene nmap es el parámetro `--script`, ya que dentro de la propia herramienta, como decía, hay una base de datos muy grande y a través del parámetro script podemos filtrar con palabras clave los scripts que nos interesan.

Por ejemplo si nos interesa filtrar por scripts que puedan afectar al protocolo FTP, podemos indicar que queremos analizar solamente el puerto 21 que es el correspondiente a FTP y además decir a nmap que queremos realizar pruebas de los scripts de su base de datos que estén relacionados únicamente con el servicio FTP. El comando sería el siguiente:

```
nmap -sV -vv -p 21 192.168.186.132 --script "*ftp*"
```

De esta forma filtramos las pruebas que queramos hacer.

Podemos concatenar también diferentes puertos que queramos analizar haciendo por ejemplo `-p 21, 80, 443`

De la misma forma pues se puede aplicar a cualquier servicio HTTP con `--script "*http*"`, puede ser a Samba con `--script "*smb*"`, según el tipo de servicio.

Debemos tener en cuenta que en el caso de usar el parámetro `--script` en un servicio con un protocolo, lo suyo es que lo filtremos también aquí en el parámetro `-p`, sería `-p 445` en el caso de Samba.

Entonces como nos indicaba previamente la herramienta, una vez tenemos un escaneo inicial realizado, podemos realizar otro escaneo mucho más intenso sobre puertos determinados y con parámetros que vayan a realizar pruebas con mayor profundidad sobre estos puertos y tener una mayor información, un mayor nivel de detalle de la configuración que puedan tener estos servicios por detrás.

Sensitive resources

Infrastructure files: robots.txt, sitemaps.xml

En cuanto a los ficheros que debemos tener en cuenta, también tenemos que pensar que hay ciertas rutas, ciertos ficheros, como son el caso de los ficheros robots.txt y sitemap.xml, que son ficheros que dan información de la arquitectura de recursos que se encuentran expuestos en la aplicación.

En el caso del fichero robots.txt, es un fichero que permite o que facilita que no se indexen rutas de una aplicación como resultados en los navegadores, de forma que si yo indico dentro del fichero robots.txt que no deseo que se indexe la ruta por ejemplo ./admin, pues los resultados si yo pongo por ejemplo en Google el nombre de la aplicación /admin, no me expondrá como resultado ese recurso ya que lo hemos indicado en el fichero robots.txt. Pero de la misma forma si lo identificamos en la aplicación cuando estamos analizando, vamos a encontrar una serie de recursos que están accesibles en la aplicación y que nos van a dar un buen conocimiento de ficheros que ya tenemos, que podemos analizar posteriormente de la propia aplicación.

Por otro lado el recurso sitemap.xml directamente lo que suele contener son el conjunto de todos los enlaces de todas las secciones que suelen tener las

aplicaciones web y de esta forma ya nos da de un primer vistazo la arquitectura o las propias secciones, las funcionalidades en las que se divide la aplicación.

Backups: backup.db, files.old, database.bak

También tenemos los tipos de ficheros que son de respaldo, de bases de datos, de copias de seguridad, que en el caso de estar visibles y con una configuración con la que tengamos acceso, podemos descargarnos en nuestro equipo local o en nuestra máquina con la que estamos realizando el auditor y tener un conocimiento de estados previos que tenía la propia aplicación. Pudiendo contener tanto usuarios antiguos, credenciales, ficheros o rutas que estuvieran en ese momento en estado de prueba o de desarrollo y que pudieran no haberse dado de baja, nos darían también información de la arquitectura y de rutas de la aplicación.

Además también otro aspecto que debemos tener muy en cuenta es que según las tecnologías, los servidores, el software que vamos a usar para auditar nuestra aplicación web, debemos tener en cuenta que según las tecnologías que usen la aplicación van a tener una arquitectura diferente, van a hacer uso de una serie de ficheros y de rutas diferentes.

En este caso podemos pensar tanto en servidores como Apache, como Nginx. O como también es el caso de CMS, como WordPress, como Drupal, como Joomla, como Prestashop, que en cada caso es necesario una estructura de ficheros diferente con unos recursos determinados y de la misma forma pasa con los sistemas operativos.

Así también esta información la podremos extraer, estas rutas concretas según la tecnología, la podemos extraer a través de ataques de fuerza bruta sobre la aplicación auditada para poder identificar estas rutas y los recursos que puedan estar disponibles.

-
- Admin interfaces
 - /wp-admin
 - /wp-content
 - /admin
 - /administration

- /support
 - /management
 - /config
 - /database
 - /mysql
 - /version
 - /kernel
 - /history
 - /installation
 - /uploads
 - /documents
 - /documentation
 - /authors
 - /tests
 - /dev
 - /temp
-

Por lo tanto, como conclusión de esta sección podríamos concluir que los servicios, cada servicio y cada puerto están identificados por un número y tienen una configuración determinada según las necesidades que haya para cada servicio.

De la misma forma, es altamente recomendable siempre habilitar en los servidores únicamente aquellos servicios y aquellos puertos que vayan a ser estrictamente necesarios para el servicio y la finalidad por los que son configurados y dados de alta. Es decir, no vamos a configurar y no vamos a tener accesibles puertos o servicios que no nos hagan falta, porque esto nos puede conllevar problemas a largo plazo y además de que a nivel de gestión de estos recursos que realmente no son necesarios, es una sobrecarga de trabajo.

Y por otra parte, también hay que tener muy en cuenta que los ficheros, los recursos, las rutas que haya expuestas en las aplicaciones, según la finalidad de estas rutas, si son de administración o de gestión o son para usuarios determinados, deben ser configuradas su autorización y su acceso según el tipo de usuario y según el tipo de fichero que se trate.

Métodos HTTP, cabeceras de seguridad y Cookies

En esta sesión vamos a comentar aspectos que debemos tener en cuenta cuando nos encontramos analizando la configuración de un servidor, pero más en concreto de un servidor web.

Para esta sesión, por lo tanto, vamos a tratar las opciones que vamos a tener que analizar a la hora de comunicarnos con servidores web y ciertos aspectos que tendremos que tener en cuenta en estos análisis de comunicación.

HTTP Methods or Verbs

Los **métodos HTTP**, también conocidos como **verbos HTTP**, son las acciones que se pueden realizar sobre un recurso disponible en un servidor web. Cada método tiene una estructura de petición diferente y un propósito específico.

- **GET**: Se utiliza para solicitar y obtener información de un recurso a través de su URI (Uniform Resource Identifier). La respuesta del servidor contendrá los datos del recurso.
- **HEAD**: Permite obtener solo los encabezados HTTP de un recurso, sin el cuerpo de la respuesta. Esto es útil para verificar la existencia de un recurso o sus metadatos sin descargar su contenido completo.
- **POST**: Se usa para enviar datos a un recurso específico en una URI para su procesamiento, como crear o actualizar información. Los datos se envían en el cuerpo de la petición.
- **PUT**: Envía datos a una URL para reemplazar o crear un recurso en esa ubicación.
- **OPTIONS**: Solicita una respuesta al servidor que enumera los métodos HTTP que están permitidos para un recurso en particular.
- **DELETE**: Se utiliza para eliminar un recurso de una aplicación. Como se puede inferir, su uso debe estar fuertemente restringido por motivos de seguridad.

HTTP Security Headers

Share additional information with an HTTP request or response.

Types:

- **Strict-Transport-Security (HSTS)**
 - Tell browsers that it should only be accessed using HTTPS, instead of using HTTP
- **X-Frame-Options**
 - Used to indicate whether a browser should be allowed to render a page in frames
 - Avoid clickjacking attacks
- **Content-Security-Policy (CSP)**
 - Helps to detect and mitigate certain types of attacks, including Cross-Site Scripting (XSS) and data injection attacks.
- **X-XSS-Protection**
 - Was introduced to protect against JavaScript injection attacks in the form of cross-site scripting.
- **X-Content-Type-Options**
 - This header forces web browsers to strictly follow the MIME types specified in Content-Type headers.

Por otro lado tenemos lo que son llamadas las cabeceras de seguridad, que son elementos que tenemos tanto en las peticiones como en las respuestas de comunicación entre cliente y servidor y comparten información adicional.

Entre las diferentes cabeceras HTTP que existen hay un conjunto que son las llamadas cabeceras HTTP de seguridad y son las que vamos a ver.

En primer lugar tenemos la cabecera Strict-Transport-Security (HSTS), lo que hace es decir a los navegadores forzar las comunicaciones a través de HTTPS en lugar de HTTP para tener un cifrado en la comunicación entre la aplicación del cliente y el servidor de la aplicación.

En siguiente lugar tenemos la cabecera X-Frame-Options que es usada principalmente para evitar que las aplicaciones sean renderizadas en frames, de forma que previene ataques de clickjacking.

También tenemos la cabecera de Content Security Policy que es conocida también como CSP, que lo que hace es ayudar a identificar y mitigar ataques de inyecciones como pueden ser también ataques de Cross site scripting, que ocurre similar con el caso de la cabecera de X-XSS-Protection que también lo que hace es ayudar a proteger contra ataques de inyección de JavaScript, que en este caso también es el mismo caso que las vulnerabilidades de Cross site scripting.

Por otra parte, en el caso de la cabecera de X-Content-Type-Options lo que hace es forzar al navegador a usar ciertos formatos de MIME type, formatos de ficheros que hay en las cabeceras del Content type.

¿Cómo vamos a comprobar si una aplicación las tiene configuradas?

Pues en este caso vamos a ver varias formas.

- Burp Suite
- Browser's network console
- Shcheck
 - source: <https://github.com/santoru/shcheck>

La primera forma va a ser a través de la aplicación de Burp Suite, de modo que vamos a irnos a nuestra máquina Kali Linux y vamos a hacer uso de la aplicación de DVWA que tenemos.

Entonces, teniendo en cuenta que tenemos nuestro proxy de Burp ya configurado, vamos a arrancar la aplicación de Burp.

Tenemos la edición Community, que es la gratuita que viene por defecto en Kali Linux y vamos a interceptar las comunicaciones entre la aplicación y el servidor.

El servidor en este caso es nuestra máquina de metasploitable.

Vamos a interceptar las comunicaciones y vamos a ver las peticiones y las respuestas que realizamos en estas comunicaciones.

Para ello vamos a la pestaña de proxy y HTTP History.

Aquí en Foxy Proxy ya hemos seleccionado para interceptar el tráfico y vamos a volver a cargar la aplicación, de forma que nuestro Burp ya la ha interceptado.

Y aquí podemos ver tanto cabeceras como puede ser la server y otras tantas.

De forma que podemos analizar las cabeceras que se encuentran en las comunicaciones y en cada uno de los recursos que hay en las aplicaciones.

Por otro lado, podemos autenticarnos con las credenciales admin / password, que si no recuerdo mal son las credenciales por defecto.

De forma que desde login.php nos redirige a index.php.

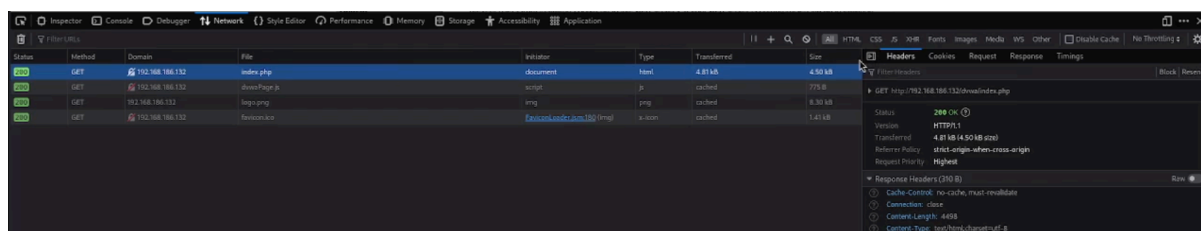
The screenshot displays the Burp Suite interface. At the top, the 'Proxy' tab is active. Below it, the 'HTTP History' tab shows a list of intercepted requests. The selected request is from 'http://192.168.186.132' to 'http://192.168.186.132' with a status of 200. The 'Request' pane shows the raw HTTP request, including headers like 'Host: 192.168.186.132', 'User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0', and 'Referer: http://192.168.186.132/dvwa/login.php'. The 'Response' pane shows the raw HTTP response, including headers like 'Date: Sun, 17 Mar 2024 18:46:11 GMT', 'Server: Apache/2.2.8 (Ubuntu)', and 'Content-Type: text/html; charset=utf-8'. The 'Inspector' pane on the right shows the HTML structure of the response, including the title 'Damn Vulnerable Web App (DVWA) v1.0.7 :: Welcome' and various links and scripts.

Lo vemos en la imagen.

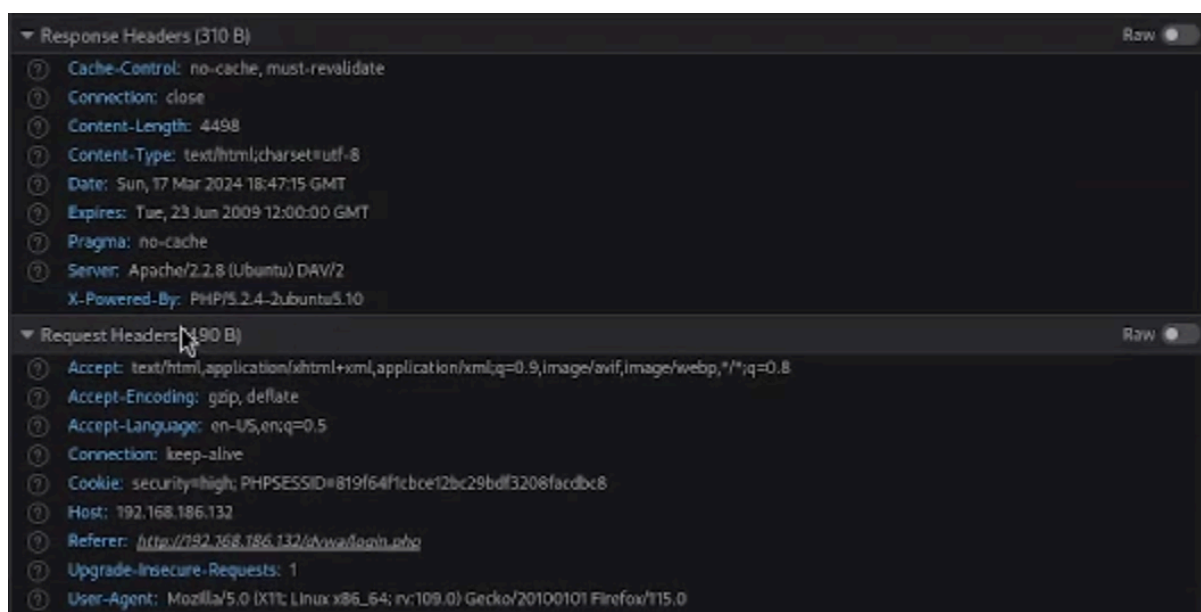
Y aquí nos cambia algunas de las cabeceras que son usadas por la aplicación.

De la misma forma hemos indicado que hay otras formas de obtener estas cabeceras, que es a través del propio navegador.

Para ello, en la aplicación web podemos hacer control derecho con el ratón, inspeccionar el elemento y en la sección de Network podemos cargar el recurso cómo vemos en la imagen.



Aquí tenemos index y vamos a ver las cabeceras que se encuentran configuradas, las cabeceras de la respuesta y por otra parte abajo nos pone las que hemos enviado nosotros a la petición. Se ve claramente en la imagen.



Además de esta técnica está la herramienta que está disponible en GitHub que se llama shcheck (security headers check) y vamos a hacer la prueba en nuestra aplicación.

Instalamos la herramienta en nuestra máquina kali linux, vamos a cambiar a la ruta donde se encuentra la aplicación y vamos a pasar la ruta donde se encuentra en este caso directamente en la URL de nuestro target. El comando en la terminal quedaría así:

```
python 3 shcheck.py http://192.168.168.186/dvwa/index.php
```

```
(root@kali)~/home/kali/Tools/shcheck
# python3 shcheck.py http://192.168.186.132/dvwa/index.php

> shcheck.py - santoru .....
Simple tool to check security headers on a webserver

[*] Analyzing headers of http://192.168.186.132/dvwa/index.php
[*] Effective URL: http://192.168.186.132/dvwa/login.php
[!] Missing security header: X-Frame-Options
[!] Missing security header: X-Content-Type-Options
[!] Missing security header: Content-Security-Policy
[!] Missing security header: Referrer-Policy
[!] Missing security header: Permissions-Policy
[!] Missing security header: Cross-Origin-Embedder-Policy
[!] Missing security header: Cross-Origin-Resource-Policy
[!] Missing security header: Cross-Origin-Opener-Policy

[!] Headers analyzed for http://192.168.186.132/dvwa/login.php
[+] There are 0 security headers
[-] There are not 8 security headers
```

Aquí en este caso, cómo vemos en la imagen, nos analiza las cabeceras HTTP que podemos ver a continuación.

Además nos redirige a login en lugar de a index, por lo tanto podemos cambiar nuestro target a login.php y de la misma forma nos va a indicar que no hay ninguna de estas cabeceras HTTP de seguridad configuradas en esta aplicación. Aquí vemos el comando y en la imagen el output.

`python 3 shcheck.py http://192.168.186.132/dvwa/login.php`

```
(root@kali)~/home/kali/Tools/shcheck
# python3 shcheck.py http://192.168.186.132/dvwa/login.php

> shcheck.py - santoru .....
Simple tool to check security headers on a webserver

[*] Analyzing headers of http://192.168.186.132/dvwa/login.php
[*] Effective URL: http://192.168.186.132/dvwa/login.php
[!] Missing security header: X-Frame-Options
[!] Missing security header: X-Content-Type-Options
[!] Missing security header: Content-Security-Policy
[!] Missing security header: Referrer-Policy
[!] Missing security header: Permissions-Policy
[!] Missing security header: Cross-Origin-Embedder-Policy
[!] Missing security header: Cross-Origin-Resource-Policy
[!] Missing security header: Cross-Origin-Opener-Policy

[!] Headers analyzed for http://192.168.186.132/dvwa/login.php
[+] There are 0 security headers
[-] There are not 8 security headers
```

Con esta aplicación podemos analizar las cabeceras HTTP de cualquier aplicación web que tengamos expuesta en Internet.

Podemos hacer la prueba con HTTPs con oxword, por ejemplo:

`python 3 shcheck.py https://www.0xword.com`

```
(root@kali)-[/home/kali/Tools/shcheck]
python3 shcheck.py https://www.0xword.com

> shcheck.py - santoru .....
Simple tool to check security headers on a webserver

[*] Analyzing headers of https://www.0xword.com
[*] Effective URL: https://0xword.com/
[!] Missing security header: X-Frame-Options
[!] Missing security header: X-Content-Type-Options
[*] Header Strict-Transport-Security is present! (Value: max-age=63072000; includeSubDomains; preload)
[!] Missing security header: Content-Security-Policy
[!] Missing security header: Referrer-Policy
[!] Missing security header: Permissions-Policy
[!] Missing security header: Cross-Origin-Embedder-Policy
[!] Missing security header: Cross-Origin-Resource-Policy
[!] Missing security header: Cross-Origin-Opener-Policy

[!] Headers analyzed for https://0xword.com/
[+] There are 1 security headers
[~] There are not 0 security headers
```

En este caso, cómo vemos en la imagen de arriba, nos indica que la cabecera HSTS sí está configurada como ejemplo de aplicación que está expuesta públicamente para cada uno de nosotros.

Continuamos con los cookies.

Conectando Conceptos: del Reconocimiento a las Cabeceras

El flujo de una auditoría web es un proceso escalonado. Primero, se realiza una **enumeración** para descubrir los servicios de un servidor y los recursos que expone. Una vez que se tiene una idea clara de la infraestructura, se pasa a un análisis más detallado del comportamiento de la aplicación. Es aquí donde entran en juego las **cabeceras de seguridad HTTP**.

Estas cabeceras son elementos esenciales que se envían tanto en las peticiones como en las respuestas, proporcionando información adicional sobre la comunicación. Su análisis es crucial porque una configuración deficiente puede exponer la aplicación a ataques, incluso si el código fuente es seguro. En la auditoría, se revisan estas cabeceras para determinar si están configuradas correctamente y si ofrecen una capa de defensa adicional.

Cabeceras de Seguridad HTTP y su Propósito

Las **cabeceras de seguridad HTTP** son directivas que se envían entre el servidor y el navegador para reforzar la seguridad. Las más comunes son:

- **Strict-Transport-Security (HSTS)**: Esta cabecera le ordena al navegador que solo se comunice con el sitio a través de **HTTPS**, en lugar de **HTTP**. Esto previene ataques de degradación de protocolo y asegura que todas las comunicaciones estén cifradas.
- **X-Frame-Options**: Se utiliza para controlar si una página web puede ser renderizada dentro de un `<iframe>`. Su principal objetivo es mitigar los ataques de *clickjacking*, donde un atacante oculta una página legítima para engañar al usuario y que haga clic en un elemento malicioso.
- **Content-Security-Policy (CSP)**: Conocida como **CSP**, es una cabecera que ayuda a prevenir ataques de inyección de código, como el **Cross-Site Scripting (XSS)**. Funciona permitiendo a los administradores del sitio especificar qué fuentes de contenido (scripts, estilos, imágenes) se consideran seguras.
- **X-XSS-Protection**: Es una cabecera más antigua que se diseñó para activar el filtro de **XSS** del navegador. Aunque sigue siendo útil, ha sido en gran medida reemplazada por **CSP**.
- **X-Content-Type-Options**: Esta cabecera fuerza a los navegadores a seguir estrictamente el tipo de contenido (**MIME type**) declarado en la respuesta. Esto previene ataques de "falsificación de tipo de contenido" donde un atacante podría engañar al navegador para que ejecute un archivo de un tipo inesperado.

Procedimiento Práctico para la Verificación de Cabeceras

Para verificar la configuración de estas cabeceras, se pueden utilizar diferentes técnicas, como la inspección manual y el uso de herramientas automatizadas.

1. Verificación con Burp Suite

1. **Configuración del Proxy**: Primero, se debe asegurar que **Burp Suite** está configurado para actuar como un proxy entre el navegador y el servidor. Esto se logra configurando el navegador para que dirija el tráfico al **localhost** en el puerto **8080**.

2. **Captura de Tráfico:** Con la función **HTTP History** de la pestaña **Proxy** en **Burp Suite** activada, se navega a la aplicación web (en este caso, **DVWA**). Todas las peticiones y respuestas se registran.
3. **Análisis de Encabezados:** Al seleccionar una petición en el historial, se puede analizar la respuesta del servidor en el panel inferior, específicamente en la vista **"Headers"**. Aquí se pueden buscar manualmente las cabeceras de seguridad para ver cuáles están presentes o ausentes. En la práctica, se observa que la aplicación **DVWA** no tiene ninguna de estas cabeceras de seguridad configuradas.

2. Verificación con el Navegador

Una alternativa a **Burp Suite** es usar las herramientas de desarrollo del propio navegador (F12).

1. **Abrir Consola:** En la página de la aplicación web, se presiona **F12** y se navega a la pestaña **Network**.
2. **Cargar el Recurso:** Al recargar la página, se muestran todas las peticiones que se han realizado. Al seleccionar una de ellas (por ejemplo, **index.php**), se pueden ver las cabeceras de la petición y de la respuesta en el panel lateral. Esto proporciona una forma rápida y sencilla de verificar si las cabeceras están presentes.

3. Verificación Automatizada con **shcheck**

Para una verificación más rápida y completa, se puede usar una herramienta automatizada como **shcheck**.

1. **Ejecutar **shcheck**:** El comando **python3 shcheck.py <URL>** realiza un análisis de las cabeceras de seguridad de la URL proporcionada.
2. **Análisis de DVWA:** Al ejecutar el comando con la URL de **DVWA** (**http://192.168.168.186/dvwa/login.php**), la herramienta confirma que no se ha encontrado ninguna cabecera de seguridad configurada.
3. **Análisis de un sitio seguro:** En contraste, al ejecutar la herramienta en un sitio público como **https://www.0xword.com**, el **output** muestra que la

cabecera `Strict-Transport-Security` está presente y bien configurada, con una directiva `max-age`, `includeSubDomains` y `preload`. Esto sirve como un claro ejemplo de una implementación segura.

Cookies

Hyper Text Transfer Protocol (HTTP)

- Stateless
- Control of state -> Set-cookie

Una vez tenemos claro las cabeceras y los métodos HTTP, debemos conocer también lo que son las cookies.

Porque en primer lugar la necesidad de las cookies viene por el propio uso y la propia definición del protocolo de HTTP.

Este protocolo se trata de un protocolo sin estado de envío y recepción de solicitudes y respuestas, de modo que no hay un control del estado de las sesiones.

Y para establecer este control del estado las opciones que pueda seleccionar un usuario una aplicación, toda esta gestión se llevan a cabo a través de las cookies, de forma que con el parámetro `set-cookie` es posible identificar y configurar ciertas opciones de seguridad que van a tener las cookies de las aplicaciones.

Por una parte vamos a tener los tres **Flags** que nos interesan:

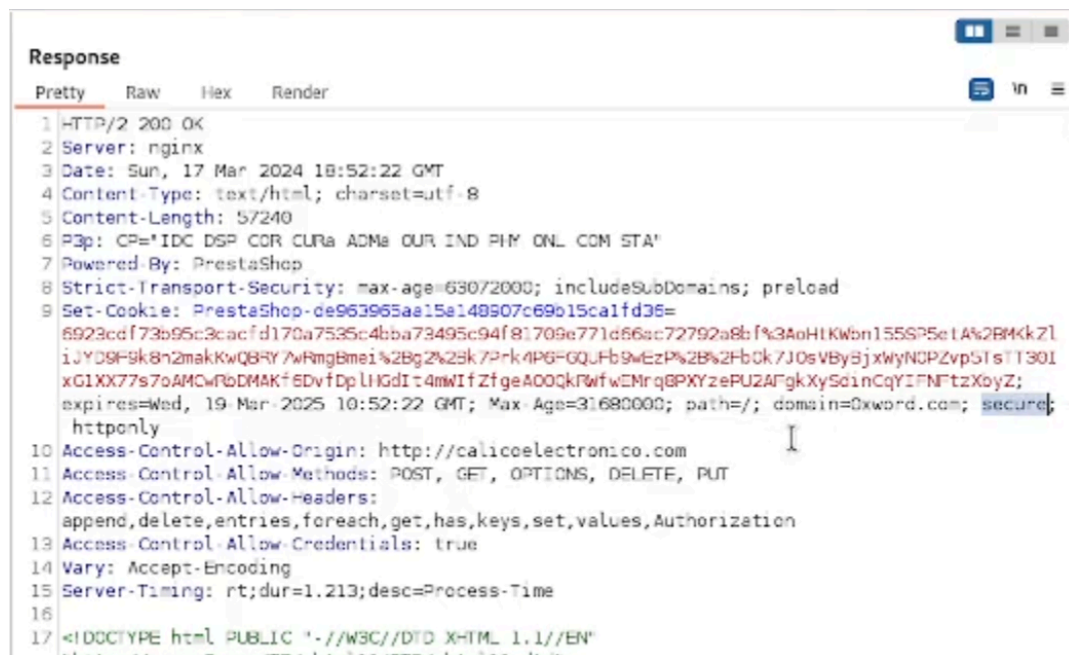
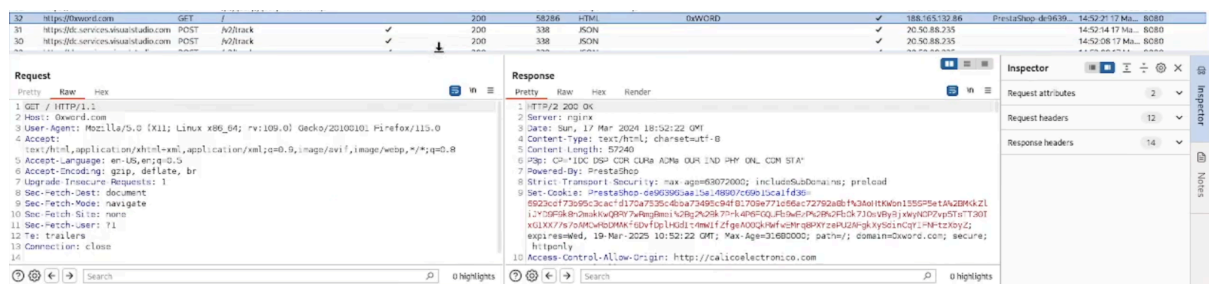
- **HttpOnly**
 - Prevent access to cookie values via JavaScript
- **Secure**
 - Forces the communications over the HTTPS protocol
- **Samesite**
 - Prevent Cross-Site Request Forgery (CSRF) attacks, que son ataques a usuarios autenticados.

De esta forma podemos volver a nuestra máquina Kali y en Burp al identificar aplicaciones en las que necesitamos autenticación.

En el caso de requerir autenticación debemos tener en cuenta que las cookies de sesión van a tener una mayor criticidad que las cookies de uso normal de una aplicación.

Pero por ejemplo podemos usar cookies la aplicación de MyPublicInbox donde vemos cómo se cargan una gran cantidad de recursos, pero en este caso no vemos cookies.

Vamos a probar otra aplicación que podría ser Oxword.com y en este caso tenemos aquí la cookie de PrestaShop donde vemos que tenemos configurados el parámetro secure y httponly. En la imagen lo tenemos más claro.



De esta forma, interceptando las comunicaciones tenemos una mayor facilidad para identificar estas configuraciones de las aplicaciones.

El Rol de las Cookies en HTTP

El protocolo **HTTP** es inherentemente "sin estado" (**stateless**). Esto significa que no tiene memoria de las peticiones anteriores del usuario. Para que una aplicación web pueda "recordar" a un usuario y mantener su sesión (por ejemplo, después de un inicio de sesión), necesita una forma de controlar este estado. Aquí es donde entran las **cookies**.

Las cookies son pequeños fragmentos de datos que el servidor envía al navegador del usuario. El navegador los almacena y los reenvía con cada petición posterior al mismo servidor, lo que permite a la aplicación identificar al usuario, mantener la sesión y personalizar la experiencia. La directiva **Set-Cookie** en la respuesta del servidor es la que se utiliza para crear y configurar una cookie.

Flags de Seguridad de las Cookies

El texto menciona tres *flags* o parámetros de seguridad que son cruciales para proteger las cookies de diversos ataques:

- **HttpOnly**: Este *flag* evita que los *scripts* del lado del cliente, como **JavaScript**, puedan acceder o manipular el valor de la cookie. Esto es una defensa fundamental contra ataques de **Cross-Site Scripting (XSS)**, ya que un atacante no podría robar la cookie de sesión de un usuario a través de un *script* inyectado.
- **Secure**: Este *flag* indica al navegador que la cookie solo debe ser enviada al servidor si la comunicación se realiza a través de un canal seguro (**HTTPS**). Esto previene que la cookie sea interceptada por un atacante en una red insegura.
- **SameSite**: Esta cabecera previene los ataques de **Cross-Site Request Forgery (CSRF)**, que ocurren cuando un atacante fuerza a un usuario autenticado a realizar acciones no deseadas en una aplicación web. **SameSite** le dice al navegador que no debe enviar la cookie si la petición proviene de un dominio diferente.

Procedimiento Práctico en Burp Suite

La fase práctica consiste en usar **Burp Suite** para inspeccionar las cookies de una aplicación web y verificar si estas cabeceras de seguridad están configuradas correctamente.

1. **Configuración de Burp:** Se asume que Burp Suite y el navegador (en este caso, Firefox) ya están configurados para interceptar el tráfico.
2. **Navegación y Captura:** Se navega a una aplicación web, como 0xword.com o mypublicinbox.com, con el proxy de Burp activado. Burp captura automáticamente todas las peticiones y respuestas.
3. **Análisis en HTTP History:** Dentro de la pestaña **Proxy**, en la sección **HTTP History**, se selecciona una de las peticiones. En el panel de **Response**, se puede ver la respuesta del servidor y, en las cabeceras, la directiva **Set-Cookie**.
 - En el caso de 0xword.com, la cabecera **Set-Cookie** muestra **secure;** y **httponly;**.
4. **Interpretación de la Salida:**
 - La presencia de **secure** indica que la cookie se transmitirá solo a través de **HTTPS**, protegiéndola de la interceptación en la red.
 - La presencia de **httponly** confirma que la cookie no puede ser accedida por **JavaScript**, lo que protege contra el robo de cookies a través de ataques **XSS**.

En resumen, la práctica demuestra que el análisis de las cabeceras y los *flags* de las cookies en herramientas como Burp Suite es una forma eficaz de auditar la seguridad de una aplicación web, especialmente en lo que respecta a la gestión de sesiones y la protección de datos sensibles.

Como conclusiones de esta sección podemos concluir que es altamente importante diferenciar los métodos HTTP según el tipo de petición o el tipo de información que queramos obtener de los recursos que hay expuestos en las aplicaciones web.

De la misma forma debemos saber y debemos diferenciar y analizar las cabeceras HTTP que se encuentran configuradas en servidores web.

Y por otra parte debemos tener en cuenta que principalmente las cookies de sesión, que son las más relevantes en el caso de las aplicaciones web, deben contener configurados los parámetros `httponly`, `secure` y `samesite` correctamente.

Ejercicio sobre configuración de servicios:

metasploitable

Vamos a explicar el ejercicio propuesto en relación a la identificación de puertos de servicios de tecnologías que se pueden encontrar configuradas en un servidor.

Para ello hemos pensado en cuatro puntos para practicar estos aspectos.

En primer lugar vamos a realizar un escaneo de los servicios expuestos en la máquina de metasploitable que cada uno de nosotros podemos tener.

De la misma forma, si en vez de la máquina metasploitable conocemos las plataformas como hack de box, podemos conectarnos a máquinas que contengan ya vulnerabilidades también y retos y realizar este tipo de pruebas sobre aquellos retos y aplicaciones que pueden puedan tener desplegadas aplicaciones web.

De la misma forma, como segundo punto vamos a buscar tres aplicaciones web que se encuentran expuestas en Internet, que sean aplicaciones o plataformas que sean conocidas y vamos a analizar las cabeceras HTTP de estas 3 páginas web.

De la misma forma vamos a observar qué métodos pueden encontrarse disponibles en estas páginas.

Y por último también podremos observar las cookies que se encuentran configuradas en estas aplicaciones.

En este aspecto de las cookies debemos tener en cuenta que las aplicaciones que las cookies a analizar en estas aplicaciones serán sin estar autenticada o autenticada en estas aplicaciones, de forma que serán las cookies configuradas cuando estableces conexión inicial con estas aplicaciones.

Resolución ejercicio sobre configuración de servicios: metasploitable

Vamos a tratar de resolver el ejercicio propuesto en relación con la identificación de puertos, servicios y tecnologías que pueden encontrarse configuradas en un servidor.

En primer lugar vamos a buscar los puertos o servicios que se encuentran habilitados y configurados en la máquina metasploitable.

En este punto quiero hacer un inciso porque también podríamos realizar esta prueba sobre máquinas de entornos tales como HacktheBox, ya que esta plataforma está también orientada al aprendizaje, a la puesta en práctica de conocimientos de ciberseguridad, es muy interesante que podamos realizar pruebas en este tipo de entornos, por ello, si tenemos o conocemos máquinas que se encuentren disponibles en este tipo de plataformas como HackTheBox, os invito a que si habéis identificado una máquina que tenga servicios web, la uséis también para realizar este ejercicio, ya que puede ser válida perfectamente.

1- Search ports and services enabled in your Metasploitable virtual machine

Vamos a identificar que tenemos nuestra máquina metasploitable habilitada, tenemos una dirección IP asignada a esta máquina de metasploitable, nuestra máquina víctima y la dirección IP está puesta como IP target para lanzar ésta prueba, es la ip 192.168.186.132. Lanzamos la prueba con el siguiente comando:

```
nmap -sV -vv -A 192.168.186.132
```

Vamos a usar tres parámetros sencillos, donde `-sV` nos va a dar la versión de los servicios que se encuentran expuestos identificados por nmap, el parámetro `-vv` va a darnos un mayor nivel de detalle de la salida de la herramienta, de forma que nos va a ir indicando con más detalle el avance que va teniendo la herramienta con las

pruebas que va realizando sobre el target, y por último el parámetro **-A** lanza una serie de scripts que tiene en nmap en su base de datos por defecto y que lanza a esta IP víctima, en los puertos y los servicios que identifica. De forma que en el caso por ejemplo del puerto 21 de FTP, a pesar de que no se trata de un puerto o de un servicio asociado a aplicaciones web como tal, también está expuesto en la máquina metasploitable y nos da información sobre este tipo de servicio.

Por otra parte vamos a buscar directamente el servicio de HTTP, que lo tenemos aquí en la imagen abajo, donde nos indica por una parte la versión del servidor, que también lo tenemos aquí como parte de una cabecera HTTP que identifica en una de las pruebas que realiza con el parámetro **a** y nos indica por otro lado los métodos HTTP que se encuentran configurados en este servidor, que nos muestra el método GET, HEAD, POST y OPTIONS.

```
80/tcp open  http          syn-ack ttl 64 Apache httpd 2.2.8 ((Ubuntu) DAV/2)
|_http-server-header: Apache/2.2.8 (Ubuntu) DAV/2
|_http-title: Metasploitable2 - Linux
|_http-methods:
|_  Supported Methods: GET HEAD POST OPTIONS
```

2- Search for the HTTP headers of 3 websites of your election:

Por otra parte también como segundo punto nos dice que tenemos que identificar las cabeceras HTTP de tres aplicaciones web que sean de nuestra elección.

Para este caso os invito a que estas páginas web sean conocidas, sean de plataformas de empresas, de organizaciones, que sean muy conocidas.

Y cómo estas pruebas no tienen impacto sobre los servidores apenas, es una forma también de conocer diferentes tipos de configuraciones en diferentes servidores.

Para ello vamos a volver a nuestra máquina Kali y en este caso vamos al navegador web. Aquí hemos elegido la aplicación de Apple, de Tesla y de Wikipedia. Son tres aplicaciones diferentes, tres páginas web independientes .

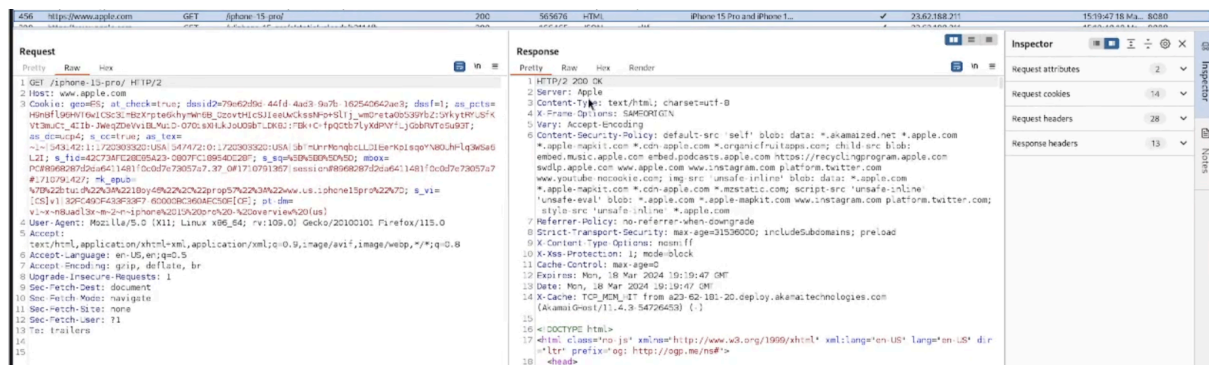
Para identificar las cabeceras HTTP, en primer lugar vamos a hacerlo a través de la aplicación de Burp suite.

Tenemos nuestro proxy ya habilitado con Foxy Proxy.

Entonces vamos a refrescar la página web de apple y vamos a irnos a la aplicación de Burp suite para seleccionar la petición que queremos y analizar los métodos y cabeceras.

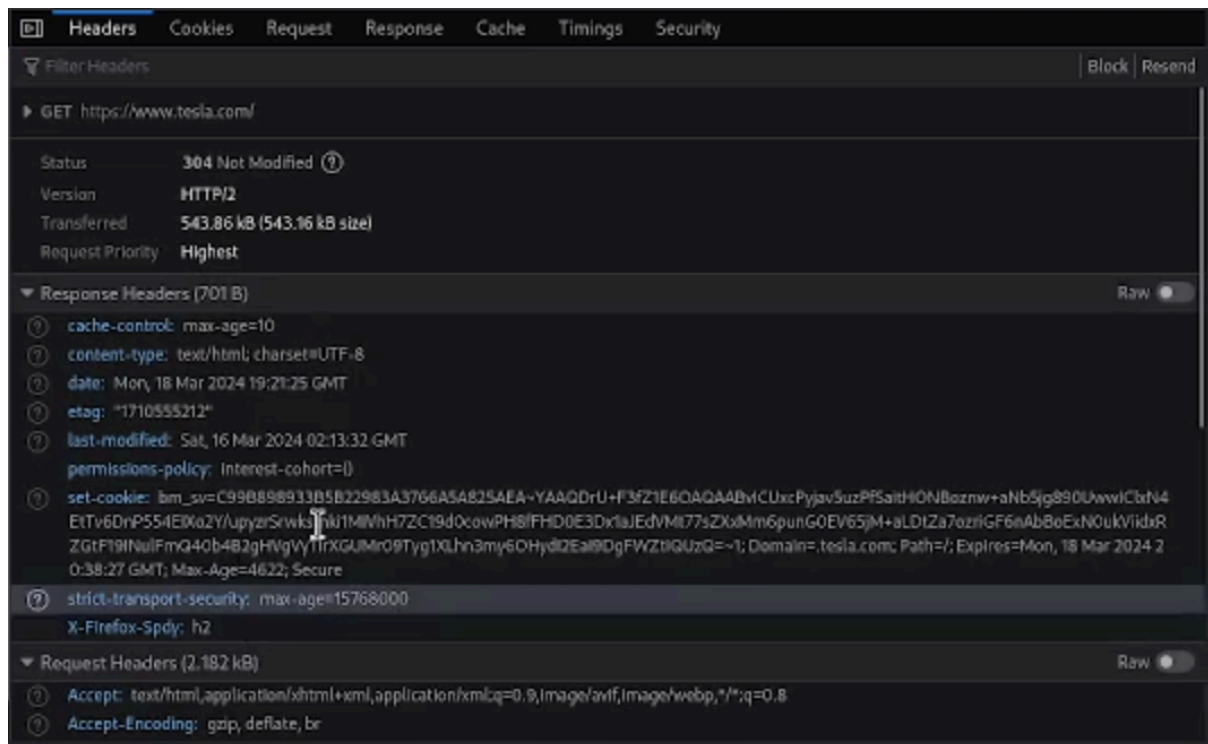
Aquí lo que tenemos es una gran cantidad de peticiones que se están haciendo a la aplicación, ya que tiene que cargar una gran variedad de recursos.

Cogemos el recurso que nos interesa analizar. Podemos ver con las cabeceras HTTP, pues que tiene por ejemplo Content-Security-Policy, que tiene nuestra cabecera HSTS, X-Content-Type-Options, X-Xss-Protection. Tiene algunas de las cabeceras que hemos visto e identificado.



Por otra parte, hemos visto también la aplicación de Tesla, la refrescamos y vamos a verlo de una forma similar pero no con burp suite sino con el inspector de elementos o f12.

Para poder ver las cabeceras HTTP recordamos que podemos hacerlo con el botón Inspeccionar, vamos a la sección de network y podemos recargar este recurso de forma que aquí en la raíz de la aplicación nos va a mostrar las cabeceras HTTP que tiene configuradas tanto de la petición como de la respuesta.



Por último, para la aplicación de Wikipedia, lo que vamos a hacer es irnos a la terminal y utilizar la herramienta shcheck, que se activa con python.

Vamos a contrastar tanto la aplicación de Apple.com donde podemos ver que tiene estas cabeceras HTTP configuradas como vamos a hacerlo también con la aplicación de Wikipedia.

Para usar shcheck con apple.com usamos éste comando:

```
python3 shcheck.py https://apple.com
```

Y nos sale éste output:



Para wikipedia usamos éste comando:

```
python3 shcheck.py
```

```
https://wikipedia.org/wiki/Wikipedia:Portada
```

Y nos sale el siguiente output:

```
root@kali:~/hacker/tools/shcheck# python3 shcheck.py https://es.wikipedia.org/wiki/Wikipedia:Portada
> shcheck.py - santoru .....
Simple tool to check security headers on a webserver

[+] Analyzing headers of https://es.wikipedia.org/wiki/Wikipedia:Portada
[+] Effective URL: https://es.wikipedia.org/wiki/Wikipedia:Portada
[+] Missing security header: X-Frame-Options
[+] Header X-Content-Type-Options is present! (Value: nosniff)
[+] Header Strict-Transport-Security is present! (Value: max-age=106384710; includeSubDomains; preload)
[+] Missing security header: Content-Security-Policy
[+] Missing security header: Referrer-Policy
[+] Missing security header: Permissions-Policy
[+] Missing security header: Cross-Origin-Embedder-Policy
[+] Missing security header: Cross-Origin-Resource-Policy
[+] Missing security header: Cross-Origin-Opener-Policy
[+] Headers analyzed for https://es.wikipedia.org/wiki/Wikipedia:Portada
[+] There are 2 security headers
[+] There are not security headers
```

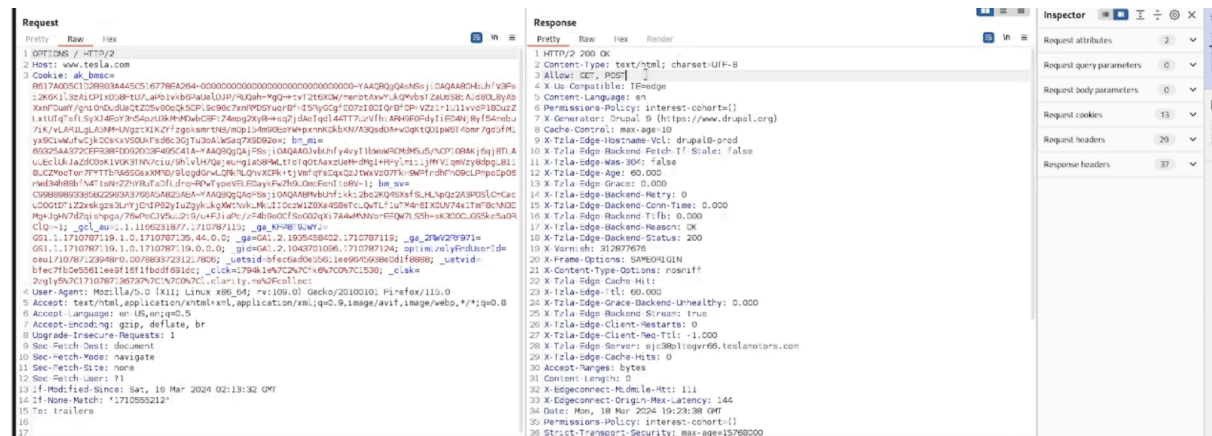
Nos indica que solamente tiene estas dos cabeceras configuradas de las que analiza la herramienta de shcheck.

3- Analyze the HTTP methods available in the 3 previous websites chosen

Como tercer punto de esta resolución de los ejercicios, vamos a ver de estas tres aplicaciones los métodos HTTP que están disponibles.

Para ello vamos a volver a nuestra máquina Kali y lo que vamos a hacer es uso de la funcionalidad de Repeater de burpsuite, para ello en este caso, ya que tenemos la petición de la raíz de la aplicación de tesla.com, vamos a mandarlo al Repeater y en el request vemos que el método es GET / HTTP/2, el método GET vamos a cambiarlo por OPTIONS a ver qué nos indica. Al final está en OPTIONS / HTTP/2.

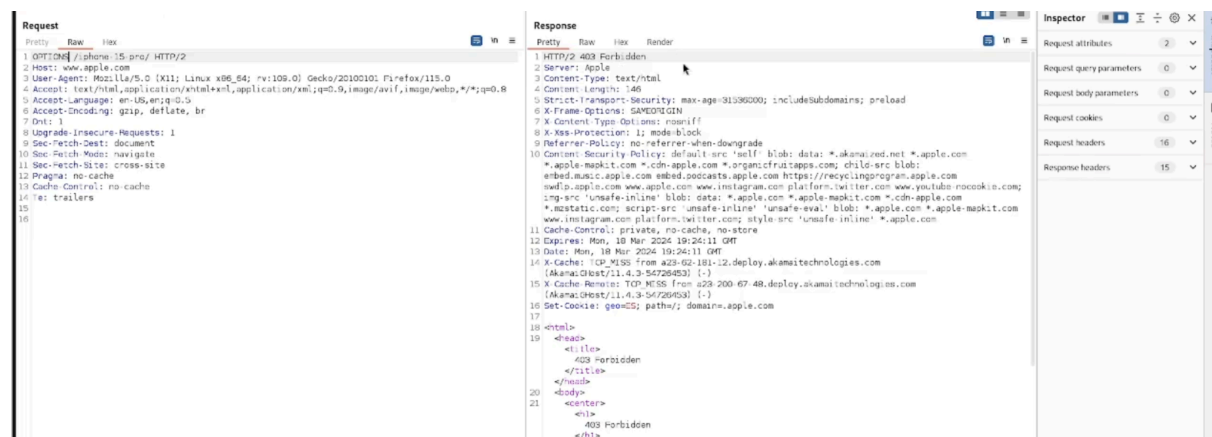
Le damos a Send.



En este caso el servidor nos dice que tiene habilitados los métodos GET y POST.

Para el caso de la aplicación de apple.com, vamos a repetir este proceso, vamos a mandar al Repeater la petición, vamos a volver a cambiar el método GET por OPTIONS en la Request. Estaba en GET / Iphone 15 pro /HTTP/2 y lo cambiamos a OPTIONS / Iphone 15 pro /HTTP/2.

En la Response nos da un error 403 forbidden. En este caso podríamos cambiar la ruta a la que se le va a solicitar el método OPTIONS, lo cambiamos a OPTIONS /HTTP/2, y también nos da un 403 Forbidden.



Debemos recordar que cada recurso dentro de la aplicación tendrá unos métodos configurados, en este caso nos da un 403, no está permitido realizar este método OPTIONS para este recurso.

De la misma forma, si por ejemplo usamos el método GET, aquí si que nos va a devolver una respuesta de 200 OK.



Por otro lado vamos a la página de Wikipedia, vamos a volver a cargarla para que la intercepte, volvemos a mandarla a Repeater y cambiamos el método por el método OPTIONS. Estaba en GET /wiki/Wikipedia:Portada HTTP/2 y lo cambiamos a OPTIONS / wiki/Wikipedia:Portada HTTP/2.



En este caso nos da otro tipo de respuesta en el que el método no está disponible (405 Method Not Allowed), nos indica el servidor de la aplicación, para ello podemos volver a intentar cambiar el recurso sobre el que queremos realizar el método OPTIONS, quitando el wiki/Wikipedia:Portada de la ruta Request como hemos hecho antes para probar.

Nos indica más de lo mismo.



Podemos probar diferentes tipos de recursos porque podemos recordar que cada recurso puede estar configurado para unos métodos u otros, probamos con OPTIONS / wiki/ HTTP/2, y nos pasa exactamente lo mismo.

Es bueno conocer este tipo de respuestas del servidor porque de esta forma tendremos un mayor conocimiento de los tipos de recursos que pueden estar configurados con unos métodos u con otros.

4- Analyze the cookies configured in the 3 previous websites

chosen:

También las cookies que se encuentran configuradas en estas tres aplicaciones, vamos a verlas en nuestra máquina Kali.

Podemos retroceder en la petición de wikipedia que hemos hecho anteriormente, volver a modificar por el método GET, obtenemos la respuesta 200 OK. Aquí por ejemplo nos muestra este Set-Cookie donde se encuentra configurada la cookie Secure.

En el caso de apple.com, en este caso muestra una cookie geo, que podemos asumir que es la geolocalización o el idioma con el que se muestra la aplicación y no tiene una mayor configuración no tiene ninguna de las tres, ninguno de los tres flags que hemos conocido como HTTPOnly, Secure o Samesite, en este caso tampoco los tiene configurados.

Y en el caso de Tesla, a pesar de que nos diga un error 304 Not Modified, nos indica estas dos cookies, donde en este caso tiene el flag httponly y en este caso flag secure.

Podemos volver, en este caso en el uso de Burpsuite podemos identificar las cookies, ya que la herramienta Burpsuite tiene una columna dedicada y es fácil filtrar e identificar las cookies de las aplicaciones.

Vemos de ésta manera que la página web de tesla tiene la cookie secure.

Análisis en profundidad:

Paso 1: Escaneo de la máquina Metasploitable

Objetivo: Identificar los puertos y servicios activos en la máquina virtual Metasploitable, que actuará como nuestro entorno de prueba vulnerable.

Procedimiento: Se utiliza la herramienta **Nmap (Network Mapper)**, una utilidad de línea de comandos estándar en ciberseguridad para el descubrimiento de redes y la auditoría de seguridad. El comando utilizado es:

```
nmap -sV -vv -A 192.168.186.132
```

- **-sV (Service Version detection):** Este parámetro ordena a Nmap que intente determinar la versión del servicio que se ejecuta en cada puerto abierto. Es

crucial para identificar servicios específicos como Apache, OpenSSH, o ProFTPD y sus versiones, lo cual es fundamental para buscar vulnerabilidades conocidas asociadas a esas versiones.

- **-vv (Very Verbose):** Aumenta el nivel de detalle en la salida de Nmap. Esto te permite ver en tiempo real el progreso del escaneo, los puertos que se están probando y la información que se va recopilando. Es útil para diagnósticos y para entender el flujo del escaneo.
- **-A (Aggressive):** Este es un parámetro compuesto que activa varias opciones de escaneo avanzadas, incluyendo la detección del sistema operativo (**-O**), la detección de la versión de los servicios (**-sV**), el escaneo de scripts predeterminados de Nmap (**--script=default**) y el traceroute (**--traceroute**). Los scripts por defecto son especialmente útiles, ya que realizan pruebas adicionales sobre los servicios detectados, como la enumeración de métodos HTTP, como se ve en tu salida para el puerto 80.

Análisis de los resultados: El escaneo reveló que la máquina Metasploitable tiene varios servicios expuestos. La prueba se centra en el puerto **80**, que es el puerto estándar para el tráfico **HTTP**. Nmap identificó que en este puerto se ejecuta el servidor web **Apache httpd 2.2.8** en un sistema operativo **Ubuntu**.

Más importante aún, los scripts de Nmap (**-A**) extrajeron información sobre las **cabeceras HTTP** y los **métodos disponibles**. Se confirmó que el servidor responde con la cabecera **Server: Apache/2.2.8 (Ubuntu) DAV/2** y que los métodos HTTP soportados son **GET, HEAD, POST y OPTIONS**. Esta información es clave para la siguiente fase de la auditoría, ya que los métodos HTTP, si no están configurados de forma segura, pueden ser un vector de ataque.

Paso 2: Análisis de cabeceras HTTP en sitios web conocidos

Objetivo: Examinar la configuración de seguridad de tres sitios web populares (Apple, Tesla y Wikipedia) a través de sus cabeceras HTTP, que son una parte fundamental de la comunicación entre el navegador y el servidor.

Procedimiento: Para llevar a cabo este análisis, se utilizaron tres métodos diferentes, demostrando la versatilidad de las herramientas disponibles:

- **Burp Suite (Apple):** Se utilizó como un proxy interceptor para capturar y analizar las peticiones y respuestas HTTP. Al recargar la página de Apple, Burp Suite capturó el tráfico y permitió inspeccionar las cabeceras de respuesta.
- **Herramientas de desarrollador del navegador (Tesla):** Se usó la función de inspector (F12) en el navegador. La pestaña "Network" (Red) mostró las cabeceras de la petición y la respuesta para el sitio de Tesla, ofreciendo una forma rápida y directa de obtener la información.
- **Herramienta `shcheck` (Apple y Wikipedia):** Se empleó una herramienta de línea de comandos, `shcheck`, que automatiza el proceso de escaneo de cabeceras de seguridad. Esta herramienta es ideal para obtener un resumen rápido de las cabeceras presentes y ausentes.

Análisis de los resultados por sitio web:

1. Apple.com:

- Se observó una configuración de seguridad robusta.
- **Content-Security-Policy (CSP):** Presente y configurada, lo que ayuda a mitigar ataques como XSS.
- **Strict-Transport-Security (HSTS):** Presente, forzando la comunicación a través de HTTPS.
- **X-XSS-Protection:** Presente con el valor `1; mode=block`, una medida de seguridad adicional contra XSS.
- **X-Content-Type-Options:** Presente con el valor `nosniff`, evitando la "adivinación" del tipo MIME.
- **X-Frame-Options:** Presente con el valor `SAMEORIGIN`, protegiendo contra el clickjacking.
- `shcheck` validó estas cabeceras, confirmando que Apple tiene una buena postura de seguridad en este aspecto.

2. Tesla.com:

- Al igual que Apple, Tesla muestra una buena configuración de seguridad.
- **Strict-Transport-Security**: Presente, asegurando que el tráfico se mantenga en HTTPS.
- También se observaron otras cabeceras como **Content-Type**, **etag**, y **date**. Aunque la imagen no muestra todas las cabeceras de seguridad, la presencia de HSTS es un fuerte indicador de que el sitio prioriza la seguridad del transporte.

3. Wikipedia.org:

- La herramienta **shcheck** reveló que la configuración de Wikipedia es menos completa en comparación con Apple.
- **Strict-Transport-Security**: Presente. Esta es la única cabecera de seguridad avanzada que **shcheck** identificó.
- **X-Content-Type-Options**: Presente con el valor **nosniff**, una cabecera crucial para evitar vulnerabilidades de "MIME-sniffing".
- **shcheck** reportó varias **cabeceras de seguridad faltantes**, como **X-Frame-Options**, **X-XSS-Protection**, y **Content-Security-Policy**.
La ausencia de estas cabeceras sugiere que Wikipedia podría ser más susceptible a ciertos ataques, como el clickjacking o XSS, en comparación con un sitio como Apple. Aunque Wikipedia utiliza otras medidas de seguridad, la falta de estas cabeceras es un punto a considerar en una auditoría.

Este análisis demuestra que, si bien todos los sitios utilizan HTTPS, el nivel de seguridad varía significativamente en función de las cabeceras de seguridad que implementan. La diferencia entre las configuraciones de Apple y Wikipedia resalta la importancia de una política de seguridad web completa.

Análisis de Métodos HTTP Disponibles

¿Por qué usar el Repeater de Burp Suite?

El Repeater de Burp Suite es una herramienta indispensable en la auditoría web por su capacidad para **modificar y reenviar peticiones HTTP manualmente**. Esto es crucial para probar diferentes escenarios de ataque o, en este caso, para auditar los métodos HTTP.

- **Manipulación de Peticiones:** A diferencia de un navegador normal que solo envía peticiones **GET** o **POST**, el Repeater te permite **cambiar el método HTTP** a cualquier otro que desees, como **OPTIONS**, **PUT**, **DELETE**, etc. Esto te da un control total sobre la petición.
- **Reenvío Rápido:** Puedes enviar la misma petición o una versión modificada tantas veces como quieras con un solo clic, sin tener que recargar la página en el navegador. Esto agiliza el proceso de prueba y error.
- **Análisis de la Respuesta:** El Repeater muestra la respuesta del servidor de forma clara y sin procesar, lo que facilita el análisis de los códigos de estado (como **200 OK** o **405 Method Not Allowed**) y las cabeceras de respuesta.

¿Por qué cambiar el método **GET** por **OPTIONS**?

El método **OPTIONS** es el estándar para **consultar las capacidades de comunicación de un servidor web para un recurso específico**. En términos sencillos, le estás preguntando al servidor: "¿Qué métodos HTTP tienes habilitados para este recurso?".

- **Enumeración de Métodos:** Al enviar una petición **OPTIONS**, la respuesta del servidor a menudo incluye una cabecera **Allow** que enumera los métodos HTTP soportados.
- **Reducción de Riesgos:** El objetivo principal de una auditoría de seguridad es identificar posibles vectores de ataque. Métodos como **PUT** o **DELETE** pueden ser muy peligrosos si están habilitados sin las protecciones adecuadas, ya que podrían permitir a un atacante subir o borrar archivos del

servidor. Usar **OPTIONS** te permite descubrir estos métodos sin realizar una acción potencialmente destructiva.

Análisis de los resultados por sitio web

1. Tesla.com:

- **Métodos Soportados:** El servidor respondió a la petición **OPTIONS** con un código de estado **200 OK** y una cabecera **Allow: GET, POST**.
- **Análisis:** Esto es una configuración segura y estándar. Tesla solo permite los métodos **GET** (para obtener recursos) y **POST** (para enviar datos), lo cual minimiza la superficie de ataque.

2. Apple.com:

- **Métodos Soportados:** Al intentar el método **OPTIONS**, el servidor respondió con un código de estado **403 Forbidden**.
- **Análisis:** El servidor de Apple no permite la petición **OPTIONS** sobre los recursos probados. Aunque esto no revela explícitamente los métodos permitidos, el **403 Forbidden** es una señal de que la configuración de seguridad del servidor está restringiendo este tipo de consultas, lo cual es una medida defensiva para evitar la enumeración de métodos por parte de un atacante.

3. Wikipedia.org:

- **Métodos Soportados:** El servidor respondió con un código de estado **405 Method Not Allowed**.
- **Análisis:** Similar a Apple, Wikipedia también restringe el uso del método **OPTIONS** en los recursos probados. Este código de respuesta es incluso más específico, ya que indica que el método solicitado (**OPTIONS**) no es compatible con el recurso de destino. Esto demuestra una configuración de seguridad bien definida para evitar la enumeración de métodos, lo que reduce la información que un atacante puede obtener.

En resumen, mientras que el servidor de Tesla respondió abiertamente a la consulta de métodos, los servidores de Apple y Wikipedia mostraron una configuración más estricta al rechazar la petición **OPTIONS**, lo que es una buena práctica de seguridad. Este tipo de respuestas le dice al atacante que no puede obtener la lista de métodos de forma tan sencilla.

Paso 4: Análisis de Cookies y sus Banderas de Seguridad

Objetivo: Examinar las cookies que los servidores de los tres sitios web configuran en la conexión inicial y determinar si tienen activadas las banderas de seguridad **HttpOnly**, **Secure** y **SameSite**. La correcta configuración de estas banderas es vital para proteger la información del usuario de ataques comunes como el robo de sesión y el CSRF (Cross-Site Request Forgery).

Procedimiento: Para llevar a cabo este análisis, te basaste en la información capturada por **Burp Suite** y en la respuesta HTTP de cada sitio web. La columna de cookies en Burp Suite simplifica la identificación de las cookies configuradas por cada aplicación.

Análisis de los resultados por sitio web:

1. Wikipedia.org:

- **Banderas de Seguridad:** Se observó que una de las cookies de Wikipedia tiene la bandera **Secure**.
- **Análisis:** La presencia de la bandera **Secure** es una buena práctica de seguridad, ya que asegura que la cookie solo se transmita a través de una conexión cifrada **HTTPS**. Esto protege la cookie de ser interceptada en texto plano por un atacante en una red no segura.

2. Apple.com:

- **Banderas de Seguridad:** Se encontró una cookie llamada **geo**, probablemente utilizada para la geolocalización o el idioma. Sin embargo, no se observaron las banderas de seguridad **HttpOnly**, **Secure** o **SameSite**.
- **Análisis:** La **ausencia de las banderas de seguridad** en la cookie **geo** es una debilidad.
 - Sin **HttpOnly**, un script malicioso (por ejemplo, en un ataque de XSS) podría acceder y robar el valor de la cookie.
 - La falta de la bandera **Secure** significa que la cookie podría ser enviada a través de una conexión HTTP no cifrada si existiera una página no segura dentro del dominio, lo que la haría vulnerable a la interceptación.
 - La falta de la bandera **SameSite** la hace susceptible a ataques CSRF.
- La configuración de esta cookie contrasta con la robusta seguridad de las cabeceras HTTP de Apple. Es un recordatorio de que la seguridad debe ser evaluada en múltiples capas de la aplicación.

3. Tesla.com:

- **Banderas de Seguridad:** Las cookies de Tesla muestran tanto la bandera **HttpOnly** como la bandera **Secure**.
- **Análisis:** La combinación de **HttpOnly** y **Secure** en las cookies de Tesla es un indicio de una sólida postura de seguridad.
 - La bandera **HttpOnly** impide que la cookie sea leída por JavaScript en el navegador, lo que es una defensa clave contra ataques XSS.
 - La bandera **Secure** garantiza que la cookie no se envíe en una conexión HTTP insegura.
- Esta configuración demuestra que Tesla ha implementado medidas para proteger las cookies de sesión contra robo y manipulación.

Conclusión del Ejercicio:

A través de estos cuatro pasos, has realizado una auditoría de seguridad web completa, pasando de la identificación de servicios básicos en un entorno de prueba

vulnerable a un análisis detallado de las configuraciones de seguridad en sitios web reales de alto tráfico.

- **Paso 1 (Metasploitable):** Identificaste un servidor web Apache con métodos HTTP comunes (**GET**, **POST**, **HEAD**, **OPTIONS**), estableciendo una línea de base para un entorno de prueba.
- **Paso 2 (Cabeceras):** Demostraste que la implementación de cabeceras de seguridad varía significativamente entre sitios web. Apple tiene una configuración muy robusta, mientras que Wikipedia, aunque segura en general, carece de algunas de estas capas de protección.
- **Paso 3 (Métodos):** Descubriste que los servidores de Apple y Wikipedia responden de forma defensiva a peticiones **OPTIONS**, mientras que Tesla responde con una lista clara de métodos, lo que resalta diferentes filosofías de seguridad.
- **Paso 4 (Cookies):** Analizaste cómo las banderas de seguridad en las cookies son una capa de protección adicional. Mientras que Tesla usa **HttpOnly** y **Secure**, Apple muestra una configuración menos estricta en una cookie específica, lo que subraya la importancia de auditar cada componente.

Este ejercicio demuestra que la seguridad web no es un estado binario (seguro/inseguro) sino un espectro de defensas en múltiples capas. Un auditor debe examinar cada una de estas capas para identificar posibles puntos débiles.